

Secure Code Review Report

Name : Arnavya Chettri

Roll No : 23BCS10119

Batch : 2027

Email : arnavya.23bcs10119@sst.scaler.com

Static Analysis Tool Used: Semgrep

FINDING 1

VULNERABILITY TITLE

SQL Injection in Ticket Search Endpoint

CLASSIFICATION

True Positive

JUSTIFICATION

The `search_tickets()` function in `models.py` concatenates user-supplied input (`term`) directly into a SQL query string using the `+` operator. Semgrep flagged this with the `sqlalchemy-execute-raw-query` rule at line 118. Since the input originates from `request.args.get("q", "")` in `app.py:76` and flows without sanitization into the SQL query, an attacker can inject arbitrary SQL. This is a True Positive because the code path is reachable via the `/tickets/search` route and is fully exploitable.

SOURCE FILE INFORMATION

File: `models.py`

Function: `search_tickets(term)`

Vulnerable Line: 117-118

SCREENSHOT OF SAST TOOL OUTPUT

3 Code Findings

```
models.py
>>> python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
    (( Blocking ))
    Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
    Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
    provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
    composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
    ORM will be a better option.
    Details: https://sg.run/2b1L

118| rows = conn.execute(sql).fetchall()

>>> python.lang.security.audit.formatted-sql-query.formatted-sql-query
    (( Blocking ))
    Detected possible formatted SQL query. Use parameterized queries instead.
    Details: https://sg.run/EKwW

136| rows = conn.execute(sql).fetchall()

>>> python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
    (( Blocking ))
    Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
    Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
    provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
    composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
    ORM will be a better option.
    Details: https://sg.run/2b1L

136| rows = conn.execute(sql).fetchall()
```

SCREENSHOT OF VULNERABLE CODE

```
Windsurf: Refactor | Explain | Generate Docstring | X
115 < def search_tickets(term):
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
119     conn.close()
120     return rows
121
```

WHY IS IT VULNERABLE?

The code builds a SQL query using string concatenation:

```
sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
```

The term variable is user-controlled from the q query parameter in /tickets/search.

An attacker can input:

- ' OR 1=1 -- → retrieves all tickets across all users
- ' UNION SELECT * FROM users -- → extracts user credentials

The sqlite3 library supports parameterized queries (using ? placeholders), but the developer chose manual string concatenation instead. This is a True Positive because the vulnerability is present in the code, reachable via an authenticated route, and no input sanitization exists.

SECURITY IMPACT

SQL Injection — an attacker can:

- Read all tickets across all users (bypassing ownership restrictions)

- Extract user credentials from the users table (including MD5 password hashes)
- Potentially modify or delete database records

RECOMMENDED REMEDIATION

Replace string concatenation with a parameterized query:

```
def search_tickets(term):  
    conn = get_connection()  
    sql = "SELECT * FROM tickets WHERE subject LIKE ?"  
    rows = conn.execute(sql, ('%' + term + '%',)).fetchall()  
    conn.close()  
    return rows
```

REFERENCES

- OWASP: SQL Injection Prevention Cheat Sheet
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command

FINDING 2

VULNERABILITY TITLE

Command Injection in Diagnostics Ping Endpoint

CLASSIFICATION

True Positive

JUSTIFICATION

The /diagnostics/ping endpoint at app.py:191 passes user-controlled input from the host query parameter directly into os.popen() without any validation or sanitization. Semgrep flagged this with the dangerous-system-call rule. The host parameter can contain shell metacharacters, making this a True Positive.

SOURCE FILE INFORMATION

File: app.py

Function: ping()
Vulnerable Line: 190-192

SCREENSHOT OF SAST TOOL OUTPUT

```
57
58 >>> python.lang.security.dangerous-system-call.dangerous-system-call
59    << Blocking >>
60    Found user-controlled data used in a system call. This could allow a malicious actor to execute
61    commands. Use the 'subprocess' module instead, which is easier to use without accidentally exposing
62    a command injection vulnerability.
63    Details: https://sg.run/k0w7
64
65    191| output = os.popen("ping -n 1 " + host).read()
66
```

SCREENSHOT OF VULNERABLE CODE

```
185 @app.route("/diagnostics/ping")
186 def ping():
187     user = current_user()
188     if not user:
189         return redirect(url_for("login"))
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "<pre>" + output + "</pre>"
193
194
```

WHY IS IT VULNERABLE?

The code at line 191 executes:

```
output = os.popen("ping -n 1 " + host).read()
```

The host parameter comes from `request.args.get("host", "127.0.0.1")` — fully user-controlled. Since the command string is built via concatenation, an attacker can inject shell commands using characters like `;`, `|`, ```, or `$()`.

Example exploit:

```
?host=127.0.0.1;cat /etc/passwd
```

This executes both the ping AND the file read command. Python's `os.popen()` executes commands through a shell (`/bin/sh` or `cmd.exe`), making injection trivially exploitable. This is a True Positive because the route requires login but does not restrict who can trigger it, and no input validation exists.

SECURITY IMPACT

Remote Code Execution (RCE). An authenticated attacker can:

- Execute arbitrary OS commands on the server

- Read sensitive files (credentials, configs, database)
- Install backdoors or pivot to internal network
- Achieve full server compromise

RECOMMENDED REMEDIATION

Use the subprocess module with a command list (avoids shell interpretation) and validate the input:

```
import subprocess
import re

@app.route("/diagnostics/ping")
def ping():
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    host = request.args.get("host", "127.0.0.1")
    # Validate: only allow IP addresses or hostnames
    if not re.match(r'^[a-zA-Z0-9\.\-]+\$', host):
        return jsonify({"error": "invalid host"}), 400
    result = subprocess.run(
        ["ping", "-c", "1", host],
        capture_output=True, text=True, timeout=5
    )
    return "<pre>" + result.stdout + "</pre>"
```

REFERENCES

- OWASP: Command Injection
- CWE-77: Improper Neutralization of Special Elements used in a Command

=====

FINDING 3

VULNERABILITY TITLE

Code Injection via eval() in SLA Score Endpoint

CLASSIFICATION

True Positive

JUSTIFICATION

The /tickets/sla-score endpoint at app.py:164 passes user-controlled input from the formula query parameter directly into Python's eval() function. Semgrep flagged this with three separate rules: user-eval, eval-injection, and eval-detected. The eval() function executes arbitrary Python code, making this a True Positive.

SOURCE FILE INFORMATION

File: app.py
Function: sla_score()
Vulnerable Line: 162-164

SCREENSHOT OF SAST TOOL OUTPUT

```
>>> python.django.security.injection.code.user-eval.user-eval
(( Blocking ))
Found user data in a call to 'eval'. This is extremely dangerous because it can enable an attacker
to execute arbitrary remote code on the system. Instead, refactor your code to not use 'eval' and
instead use a safe library for the specific functionality you need.
Details: https://sg.run/PJDW

162: formula = request.args.get("formula", "1+1")
163: try:
164:     score = eval(formula)
165: except Exception as exc:
166:     return jsonify({"error": str(exc)}), 400

>>> python.flask.security.injection.user-eval.eval-injection
(( Blocking ))
Detected user data flowing into eval. This is code injection and should be avoided.
Details: https://sg.run/5QpX

164: score = eval(formula)

>>> python.lang.security.audit.eval-detected.eval-detected
(( Blocking ))
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
content can be input from outside the program, this may be a code injection vulnerability. Ensure
evaluated content is not definable by external sources.
Details: https://sg.run/ZvrD

164: score = eval(formula)
```

SCREENSHOT OF VULNERABLE CODE

```

157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     ✨ formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
167     return jsonify({"formula": formula, "score": score})
168

```

WHY IS IT VULNERABLE?

The code retrieves a formula string from the user via:

```
formula = request.args.get("formula", "1+1")
```

and passes it directly to `eval()` at line 164:

```
score = eval(formula)
```

Python's `eval()` executes arbitrary Python expressions in the current interpreter context. An attacker can provide input such as:

- `__import__('os').system('id')` → execute OS commands
- `__import__('os').system('rm -rf /')` → destructive commands
- `open('/etc/passwd').read()` → read sensitive files

The try-except block catches exceptions like `SyntaxError` but does nothing to prevent exploitation — it only returns the error message. This is a True Positive because `eval()` is called on unsanitized user input with no allowlist or restrictions.

SECURITY IMPACT

Remote Code Execution (RCE). An authenticated attacker can:

- Execute arbitrary Python code on the server
- Execute arbitrary OS commands via `os.system()` or `subprocess`
- Read, modify, or delete any file the process has access to
- Achieve full server compromise

RECOMMENDED REMEDIATION

Never use `eval()` with user input. Replace with a predefined mapping of allowed formulas or use a safe expression parser:

```
ALLOWED_FORMULAS = {"1+1": 2, "2+2": 4, "3+3": 6}
```

```
formula = request.args.get("formula", "1+1")
score = ALLOWED_FORMULAS.get(formula)
if score is None:
    return jsonify({"error": "unsupported formula"}), 400
```

For complex calculations, use a library like numexpr that only supports mathematical expressions without allowing arbitrary code execution.

REFERENCES

- OWASP: Code Injection
- CWE-94: Improper Control of Generation of Code (Code Injection)

FINDING 4

VULNERABILITY TITLE

Path Traversal in Download Attachment Endpoint

CLASSIFICATION

True Positive

JUSTIFICATION

The `/tickets/<ticket_id>/attachments/<path:filename>` endpoint at `app.py:120` constructs the file path using simple string concatenation (`ATTACHMENTS_DIR + "/" + filename`) without validating that the resolved path stays within the intended directory. Unlike `preview_attachment()` at line 131 which uses `resolve_within()` for path traversal protection, `download_attachment()` has no such safeguard. This is a True Positive.

SOURCE FILE INFORMATION

File: app.py
Function: download_attachment(ticket_id, filename)
Vulnerable Line: 120

SCREENSHOT OF SAST TOOL OUTPUT

```
>>> python.django.security.injection.code.user-eval.user-eval
(( Blocking ))
Found user data in a call to 'eval'. This is extremely dangerous because it can enable an attacker
to execute arbitrary remote code on the system. Instead, refactor your code to not use 'eval' and
instead use a safe library for the specific functionality you need.
Details: https://sg.run/PJDW

162: formula = request.args.get("formula", "1+1")
163: try:
164:     score = eval(formula)
165: except Exception as exc:
166:     return jsonify({"error": str(exc)}), 400

>>> python.flask.security.injection.user-eval.eval-injection
(( Blocking ))
Detected user data flowing into eval. This is code injection and should be avoided.
Details: https://sg.run/5QpX

164: score = eval(formula)

>>> python.lang.security.audit.eval-detected.eval-detected
(( Blocking ))
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
content can be input from outside the program, this may be a code injection vulnerability. Ensure
evaluated content is not definable by external sources.
Details: https://sg.run/ZvrD

164: score = eval(formula)
```

SCREENSHOT OF VULNERABLE CODE

```
15 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
16 def download_attachment(ticket_id, filename):
17     user = current_user()
18     if not user:
19         return redirect(url_for("login"))
20     target = ATTACHMENTS_DIR + "/" + filename
21     if not os.path.exists(target):
22         return "File not found", 404
23     return send_file(target)
```

WHY IS IT VULNERABLE?

Line 120 constructs the target path as:

```
target = ATTACHMENTS_DIR + "/" + filename
```

The filename parameter comes from the URL path (<path:filename>) and is fully user-controlled. An attacker can supply ../../internal_notes.txt to read files outside the attachments directory.

Compare with preview_attachment() at line 131 which correctly uses resolve_within() — a function in utils.py that resolves the path and checks it stays within ATTACHMENTS_DIR:

```
def resolve_within(base_dir, filename):
    base = Path(base_dir).resolve()
    target = (base / filename).resolve()
    if not str(target).startswith(str(base)):
        return None
    return target
```

The developer added path traversal protection to `preview_attachment()` but forgot to apply the same protection to `download_attachment()`. This inconsistency makes it a True Positive.

SECURITY IMPACT

Path Traversal / Arbitrary File Read. An authenticated attacker can:

- Read any file the application process has access to (e.g., `internal_notes.txt`, `/etc/passwd`)
- Read the SQLite database file `supportdesk.db` containing all user data (including MD5 password hashes)
- Read application source code to find further vulnerabilities

RECOMMENDED REMEDIATION

Use the existing `resolve_within()` function (same as `preview_attachment()`):

```
@app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
def download_attachment(ticket_id, filename):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    target = resolve_within(ATTACHMENTS_DIR, filename)
    if target is None or not target.exists():
        abort(404)
    return send_file(target)
```

REFERENCES

- OWASP: Path Traversal
- CWE-22: Improper Limitation of a Pathname to a Restricted Directory
- CWE-23: Relative Path Traversal

FINDING 5

VULNERABILITY TITLE

Path Traversal in Upload Attachment Endpoint

CLASSIFICATION

True Positive

JUSTIFICATION

The /tickets/<ticket_id>/upload endpoint at app.py:150 uses the user-supplied filename from uploaded.filename directly in os.path.join() to construct the save path. An attacker can craft a filename containing ../ sequences to write files outside the intended attachments directory. This is a True Positive.

SOURCE FILE INFORMATION

File: app.py
Function: upload_attachment(ticket_id)
Vulnerable Line: 150

SCREENSHOT OF SAST TOOL OUTPUT

```
164| score = eval(formula)

>>> python.lang.security.dangerous-system-call.dangerous-system-call
(( Blocking ))
Found user-controlled data used in a system call. This could allow a malicious actor to execute
commands. Use the 'subprocess' module instead, which is easier to use without accidentally exposing
a command injection vulnerability.
Details: https://sg.run/k0W7

191| output = os.popen("ping -n 1 " + host).read()

>>> python.flask.security.audit.directly-returned-format-string.directly-returned-format-string
(( Blocking ))
Detected Flask route directly returning a formatted string. This is subject to cross-site scripting
if user input can reach the string. Consider using the template engine instead and rendering pages
with 'render_template()'.
Details: https://sg.run/Zv6o

192| return "<pre>" + output + "</pre>"

>>> python.django.security.injection.raw-html-format.raw-html-format
(( Blocking ))
Detected user input flowing into a manually constructed HTML string. You may be accidentally
bypassing secure methods of rendering HTML by manually constructing HTML and this could create a
cross-site scripting vulnerability, which could let attackers steal sensitive user data. To be sure
this is safe, check that the HTML is rendered safely. Otherwise, use templates
('django.shortcuts.render') which will safely render HTML instead.
Details: https://sg.run/oYj1

192| return "<pre>" + output + "</pre>"
```

SCREENSHOT OF VULNERABLE CODE

```

@app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
def upload_attachment(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    uploaded = request.files.get("file")
    if not uploaded:
        return jsonify({"error": "no file"}), 400
    data = uploaded.read()
    fingerprint = fingerprint_bytes(data)
    existing = models.find_attachment_by_fingerprint(fingerprint)
    if existing:
        return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
    save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
    with open(save_path, "wb") as f:
        f.write(data)
    models.add_attachment(ticket_id, uploaded.filename, fingerprint)
    return jsonify({"message": "uploaded", "filename": uploaded.filename})

```

WHY IS IT VULNERABLE?

Line 150 constructs the save path as:

```
save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
```

The `uploaded.filename` property is provided by the client (the HTTP request) and can contain path traversal sequences like `../`. An attacker can:

- Upload a file with filename `../../evil.py` to write outside the attachments directory
- Overwrite existing application files if the process has write permissions
- Upload a file with a malicious name to confuse application logic

While `os.path.join()` does normalize paths, it doesn't prevent traversal. For example, `os.path.join("/safe/dir", "../../etc/passwd")` resolves to `/etc/passwd`. Unlike `download_attachment`, there is no `resolve_within()` check here either. This is a True Positive because the attacker controls the filename and it flows directly into a file write operation.

SECURITY IMPACT

Arbitrary File Write. An authenticated attacker can:

- Write files to arbitrary locations the server process can write to
- Overwrite application files (e.g., templates, Python files) leading to code execution
- Plant malicious files (e.g., a webshell) in the application directory
- Corrupt the application's file storage

RECOMMENDED REMEDIATION

Sanitize the filename and validate the resolved path:

```

import os

@app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
def upload_attachment(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    uploaded = request.files.get("file")
    if not uploaded:
        return jsonify({"error": "no file"}), 400

    # Sanitize filename: remove path separators
    safe_filename = os.path.basename(uploaded.filename)
    if not safe_filename:
        return jsonify({"error": "invalid filename"}), 400

    data = uploaded.read()
    fingerprint = fingerprint_bytes(data)
    existing = models.find_attachment_by_fingerprint(fingerprint)
    if existing:
        return jsonify({"message": "duplicate of existing attachment",
                        "filename": existing["filename"]})

    save_path = os.path.join(ATTACHMENTS_DIR, safe_filename)
    # Double-check resolved path stays within ATTACHMENTS_DIR
    if not str(Path(save_path).resolve()).startswith(
        str(Path(ATTACHMENTS_DIR).resolve())
    ):
        return jsonify({"error": "invalid path"}), 400

    with open(save_path, "wb") as f:
        f.write(data)
    models.add_attachment(ticket_id, safe_filename, fingerprint)
    return jsonify({"message": "uploaded", "filename": safe_filename})

```

REFERENCES

- OWASP: Path Traversal
- CWE-22: Improper Limitation of a Pathname to a Restricted Directory
- CWE-23: Relative Path Traversal

FINDING 6

VULNERABILITY TITLE

Stored Cross-Site Scripting (XSS) in Ticket Comments

CLASSIFICATION

True Positive

JUSTIFICATION

The ticket.html template at line 35 renders comment bodies using the |safe Jinja2 filter, which disables HTML escaping. User-supplied comment content from the POST to /tickets/<id>/comment is stored in the database and then rendered unsanitized. Semgrep did not flag this specific template issue (its HTML rules are Django-focused), but manual code review confirms it.

SOURCE FILE INFORMATION

File: templates/ticket.html
Function: N/A (Jinja2 template rendering)
Vulnerable Line: 35

SCREENSHOT OF SAST TOOL OUTPUT

```
templates/ticket.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
(( Blocking ))
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/N0Bp

14 | <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15 |   <input type="hidden" name="cost" value="1">
16 |   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17 | </form>
   | .....
25 | <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-
   | data">
26 |   <input type="file" name="file">
27 |   <button type="submit" class="btn-secondary">Upload attachment</button>
28 | </form>
   | .....
39 | <form method="post" action="/tickets/{{ ticket.id }}/comment">
40 |   <div class="field">
41 |     <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42 |   </div>
43 |   <button type="submit">Add comment</button>
44 | </form>
```

SCREENSHOT OF VULNERABLE CODE

```

31 <div class="card">
32   <h2>Comments</h2>
33   {% for c in comments %}
34     <div class="comment">
35       <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
36     </div>
37   {% endfor %}

```

WHY IS IT VULNERABLE?

Line 35 renders comment body content as:

```
{{ c.body|safe }}
```

The `|safe` filter tells Jinja2 to NOT escape HTML characters. This means any HTML or JavaScript in a comment will be executed in the browser of anyone viewing that ticket.

The comment is stored in the database via:

```

body = request.form.get("body", "")
models.add_comment(ticket_id, user["username"], body)

```

No sanitization or encoding is applied before storage. When another user views the ticket, the malicious script runs in their browser context.

Example attack:

Attacker posts comment: `<script>fetch('/tickets?cookie='+document.cookie)</script>`
 Every user who views that ticket has their session cookie exfiltrated.

This is a True Positive because the `|safe` filter is explicitly used and the comment body is user-controlled via an HTTP POST request.

SECURITY IMPACT

Stored Cross-Site Scripting (XSS). An attacker can:

- Steal session cookies of other users (including admins)
- Perform actions on behalf of other users (session hijacking)
- Deface the application for other users
- Redirect users to malicious sites

RECOMMENDED REMEDIATION

Remove the `|safe` filter from the template. Jinja2 auto-escapes HTML by default, so the safe filter is unnecessary:

```
<span class="comment-author">{{ c.author }}</span>{{ c.body }}
```

If HTML in comments is a required feature, use a sanitization library like

bleach to allow only safe HTML tags before storing:

```
import bleach
sanitized_body = bleach.clean(body, tags=['b', 'i', 'u', 'a'],
                              attributes={'a': ['href']})
models.add_comment(ticket_id, user["username"], sanitized_body)
```

REFERENCES

- OWASP: Cross-Site Scripting (XSS)
- CWE-79: Improper Neutralization of Input During Web Page Generation
- OWASP: XSS Prevention Cheat Sheet

FINDING 7

VULNERABILITY TITLE

Command Injection in Diagnostics Ping-Internal Endpoint

CLASSIFICATION

False Positive

JUSTIFICATION

Semgrep flagged app.py:203 with dangerous-system-call because user-controlled data flows into os.popen(). However, line 201 validates the input using is_valid_ip() which calls ipaddress.ip_address(). This function strictly validates that the input is a literal IPv4 or IPv6 address — no shell metacharacters are permitted in valid IP addresses. The injection vector is effectively blocked by this validation gate, making this a False Positive.

SOURCE FILE INFORMATION

File: app.py
Function: ping_internal()
Vulnerable Line: 203 (flagged)

Mitigating Line: 201 (is_valid_ip check)

SCREENSHOT OF SAST TOOL OUTPUT

```
>>> python.lang.security.dangerous-system-call.dangerous-system-call
(( Blocking ))
Found user-controlled data used in a system call. This could allow a malicious actor to execute
commands. Use the 'subprocess' module instead, which is easier to use without accidentally exposing
a command injection vulnerability.
Details: https://sg.run/k0W7

203: output = os.popen("ping -n 1 " + host).read()
```

SCREENSHOT OF VULNERABLE CODE

```
@app.route("/diagnostics/ping-internal")
def ping_internal():
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    host = request.args.get("host", "127.0.0.1")
    ✨ if not is_valid_ip(host):
        return jsonify({"error": "host must be a literal IP address"}), 400
    output = os.popen("ping -n 1 " + host).read()
    return "<pre>" + output + "</pre>"
```

WHY IS IT VULNERABLE?

The code at line 203 executes:

```
output = os.popen("ping -n 1 " + host).read()
```

This has the same pattern as Finding 2 — string concatenation into a shell command. However, the crucial difference is line 201:

```
if not is_valid_ip(host):
    return jsonify({"error": "host must be a literal IP address"}), 400
```

The `is_valid_ip()` function in `utils.py` uses Python's `ipaddress.ip_address()`, which only accepts valid IPv4 and IPv6 addresses:

```
def is_valid_ip(value):
    try:
        ipaddress.ip_address(value)
        return True
    except ValueError:
        return False
```

Valid IP addresses (e.g., "192.168.1.1", "::1", "2001:db8::1") cannot contain shell metacharacters like `;`, `|`, ```, or `$()`. Therefore command injection is not possible despite the dangerous pattern.

Why is this a False Positive?

- Semgrep performs static taint analysis — it sees user input flowing into `os.popen` and flags it, but it does not evaluate the strength of the intervening validation control.
- Manual review confirms the `is_valid_ip()` check is a strong defense that completely blocks injection.
- The code is still not ideal (`os.popen` with string concat is risky), but as written, it is not exploitable.

SECURITY IMPACT

None for command injection (False Positive). The `is_valid_ip()` check prevents exploitation. Note: the raw HTML return at line 204 is a separate XSS concern covered in another finding.

RECOMMENDED REMEDIATION

Since this is a False Positive, no remediation is strictly required for the command injection finding. However, for defense-in-depth, it is recommended to refactor to `subprocess` with a command list (same fix as Finding 2):

```
import subprocess

@app.route("/diagnostics/ping-internal")
def ping_internal():
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    host = request.args.get("host", "127.0.0.1")
    if not is_valid_ip(host):
        return jsonify({"error": "host must be a literal IP address"}), 400
    result = subprocess.run(
        ["ping", "-c", "1", host],
        capture_output=True, text=True, timeout=5
    )
    return "<pre>" + result.stdout + "</pre>"
```

REFERENCES

- CWE-77: Improper Neutralization of Special Elements used in a Command
- OWASP: Command Injection
- OWASP: Input Validation Cheat Sheet

FINDING 8

VULNERABILITY TITLE

Weak Password Hashing Using MD5

CLASSIFICATION

True Positive

JUSTIFICATION

The `hash_password()` function in `utils.py:7` uses MD5 (`hashlib.md5()`) for password hashing. MD5 is a cryptographically broken hash algorithm that can be cracked rapidly using commodity hardware (billions of hashes per second). Semgrep flagged this with `md5-used-as-password` rule at `utils.py:7`. This is a True Positive.

SOURCE FILE INFORMATION

File: `utils.py`
Function: `hash_password(password)`
Vulnerable Line: 7

SCREENSHOT OF SAST TOOL OUTPUT

```
225 | utils.py
226 | >> python.lang.security.audit.md5-used-as-password.md5-used-as-password
227 |   << Blocking >>
228 |   It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because
229 |   it can be cracked by an attacker in a short amount of time. Use a suitable password hashing function
230 |   such as scrypt. You can use `hashlib.scrypt`.
231 |   Details: https://sg.run/5DwD
232 |
233 |   7| return hashlib.md5(password.encode()).hexdigest()
234 |
```

SCREENSHOT OF VULNERABLE CODE

```
6 | def hash_password(password):
7 |     ⚡ return hashlib.md5(password.encode()).hexdigest()
8 |
```

WHY IS IT VULNERABLE?

Line 7 implements password hashing as:

```
return hashlib.md5(password.encode()).hexdigest()
```

MD5 is a fast, general-purpose hash designed for checksums, not password storage. Key problems:

- Speed: MD5 computes ~10 billion hashes/second on modern GPUs, making brute-force attacks trivial
- No salt: Every user with the same password gets the same hash (rainbow table vulnerability)
- Cryptographically broken: MD5 collision attacks have been demonstrated since 2004
- No work factor: Unlike bcrypt/scrypt/argon2, MD5 has no configurable cost parameter to slow down attacks

The password hashes are stored in the users table (alice, bob, carol with predictable passwords like "alice123") and are exposed if an attacker gains database access through the SQL injection finding (Finding 1).

This is a True Positive because the hashing method is fundamentally unsuitable for password storage and directly undermines authentication security.

SECURITY IMPACT

Authentication Bypass / Credential Disclosure. An attacker who gains database access (e.g., via SQL injection) can:

- Crack all password hashes in minutes using GPU-based brute force
- Use cracked credentials to log in as any user, including admin (carol)
- Escalate privileges from regular user to admin

RECOMMENDED REMEDIATION

Replace MD5 with a dedicated password hashing function:

```
import hashlib
import os

def hash_password(password):
    # Generate random 16-byte salt
    salt = os.urandom(16)
    # Use PBKDF2 with 600,000 iterations (SHA-256)
    key = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode(),
        salt,
        600000
    )
```

```
# Store salt + hash together
return salt.hex() + ':' + key.hex()

def verify_password(password, password_hash):
    salt_hex, key_hex = password_hash.split(':')
    salt = bytes.fromhex(salt_hex)
    key = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode(),
        salt,
        600000
    )
    return key.hex() == key_hex
```

Or better, use a library like bcrypt:

```
import bcrypt

def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

def verify_password(password, password_hash):
    return bcrypt.checkpw(password.encode(), password_hash.encode())
```

REFERENCES

- OWASP: Password Storage Cheat Sheet
- CWE-916: Use of Password Hash With Insufficient Computational Effort
- CWE-327: Use of a Broken or Risky Cryptographic Algorithm

FINDING 9

VULNERABILITY TITLE

Hardcoded Flask SECRET_KEY

CLASSIFICATION

True Positive

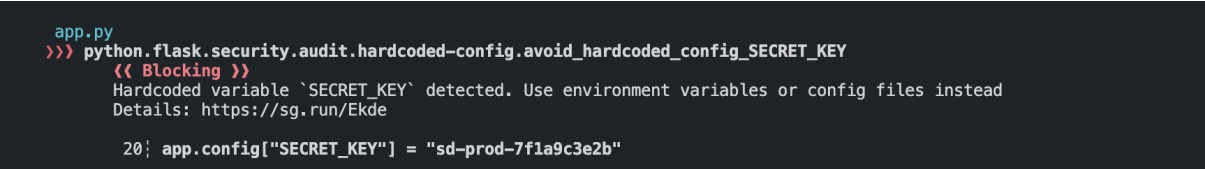
JUSTIFICATION

The Flask SECRET_KEY is hardcoded as "sd-prod-7f1a9c3e2b" in app.py:20. This key is used by Flask to sign session cookies. An attacker who obtains this key can forge arbitrary session cookies, including setting any user_id and role. Semgrep flagged this with avoid_hardcoded_config_SECRET_KEY rule. This is a True Positive.

SOURCE FILE INFORMATION

File: app.py
Function: N/A (application configuration)
Vulnerable Line: 20

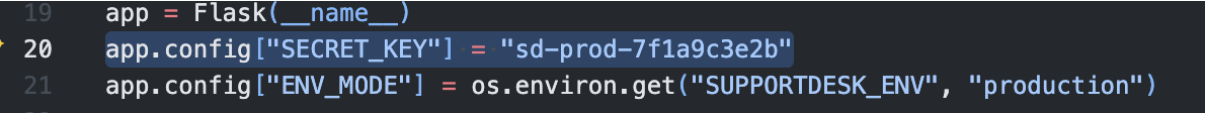
SCREENSHOT OF SAST TOOL OUTPUT



```
app.py
>>> python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY
(( Blocking ))
Hardcoded variable `SECRET_KEY` detected. Use environment variables or config files instead
Details: https://sg.run/Ekde

20: app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
```

SCREENSHOT OF VULNERABLE CODE



```
19 app = Flask(__name__)
20 app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
21 app.config["ENV_MODE"] = os.environ.get("SUPPORTDESK_ENV", "production")
```

WHY IS IT VULNERABLE?

Line 20 sets the SECRET_KEY as a hardcoded string literal:

```
app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"
```

Flask uses this key to cryptographically sign session cookies. If an attacker knows the SECRET_KEY, they can:

- Decode and modify session cookies
- Set session["user_id"] to any value (e.g., 3 for admin carol)
- Set session["role"] to "admin"
- Forge a valid session without knowing any password

The key is stored in plaintext in the source code, accessible to anyone who can read the application files (including via the path traversal finding). This is a True Positive because the key is static, hardcoded, and shared across all deployments.

SECURITY IMPACT

Authentication Bypass / Privilege Escalation. An attacker who obtains the SECRET_KEY can:

- Forge session cookies to impersonate any user
- Escalate privileges to admin by setting session["role"] = "admin"
- Bypass authentication entirely

RECOMMENDED REMEDIATION

Move the SECRET_KEY to an environment variable:

```
import os

app.config["SECRET_KEY"] = os.environ.get(
    "SUPPORTDESK_SECRET_KEY",
    os.urandom(32).hex() # fallback for development only
)
```

Then set the environment variable in production:

```
export SUPPORTDESK_SECRET_KEY="$(python3 -c 'import os;
print(os.urandom(32).hex())')"
```

REFERENCES

- OWASP: Cryptographic Storage Cheat Sheet
- CWE-798: Use of Hard-coded Credentials
- CWE-321: Use of Hard-coded Cryptographic Key

FINDING 10

VULNERABILITY TITLE

Debug Mode Enabled in Production

CLASSIFICATION

True Positive

JUSTIFICATION

The Flask application is started with `debug=True` and `host="0.0.0.0"` at `app.py:220`. Flask debug mode enables an interactive debugger that allows executing arbitrary Python code through the browser. Semgrep flagged this with both `debug-enabled` and `avoid_app_run_with_bad_host` rules. This is a True Positive.

SOURCE FILE INFORMATION

File: `app.py`
Function: N/A (application entry point)
Vulnerable Line: 220

SCREENSHOT OF SAST TOOL OUTPUT

```
>> python.flask.security.audit.app-run-param-config.void_app_run_with_bad_host
  << Blocking >>
  Running flask app with host 0.0.0.0 could expose the server publicly.
  Details: https://sg.run/eLby

220: app.run(debug=True, host="0.0.0.0", port=5050)

>> python.flask.security.audit.debug-enabled.debug-enabled
  << Blocking >>
  Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will
  leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug'
  using system environment variables.
  Details: https://sg.run/dKrd

220: app.run(debug=True, host="0.0.0.0", port=5050)
```

SCREENSHOT OF VULNERABLE CODE

```
if __name__ == "__main__":
    models.init_db()
    if not os.path.exists(ATTACHMENTS_DIR):
        os.makedirs(ATTACHMENTS_DIR)
    app.run(debug=True, host="0.0.0.0", port=5050)
```

WHY IS IT VULNERABLE?

Line 220 starts the application with debug mode enabled on all network interfaces:

```
app.run(debug=True, host="0.0.0.0", port=5050)
```

Flask debug mode provides:

- An interactive debugger (Werkzeug debugger) that allows executing arbitrary Python code in the browser when an exception occurs
- A reloader that auto-restarts on code changes
- Detailed stack traces with full source code disclosure

The combination of `debug=True` and `host="0.0.0.0"` means the debugger is accessible from any network interface, not just localhost. An attacker who triggers an exception (e.g., by sending malformed input) gains access to an interactive Python console on the server.

This is a True Positive because the application is configured to bind to all interfaces in debug mode, which is never appropriate for production.

SECURITY IMPACT

Remote Code Execution / Information Disclosure. An attacker can:

- Access the Werkzeug debugger console to execute arbitrary Python code
- View detailed stack traces with source code, environment variables, and file contents
- Achieve full server compromise through the debugger console
- Read sensitive application data through exposed tracebacks

RECOMMENDED REMEDIATION

Remove `debug=True` in production and bind only to localhost for development:

```
if __name__ == "__main__":
    models.init_db()
    if not os.path.exists(ATTACHMENTS_DIR):
        os.makedirs(ATTACHMENTS_DIR)

    debug_mode = os.environ.get("SUPPORTDESK_ENV") == "development"
    app.run(debug=debug_mode, host="127.0.0.1", port=5050)
```

REFERENCES

- OWASP: Security Misconfiguration
- CWE-489: Active Debug Code
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

FINDING 11

VULNERABILITY TITLE

Missing Access Control — Any User Can View Any Ticket

CLASSIFICATION

True Positive

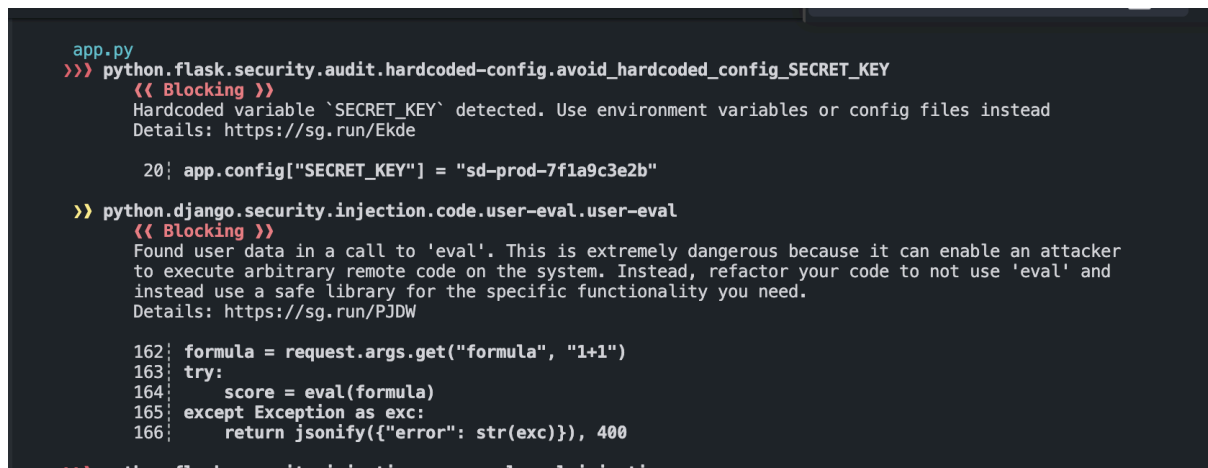
JUSTIFICATION

The `view_ticket()` function at `app.py:95-98` checks that the user is authenticated (logged in) but does NOT verify that the ticket belongs to that user. Any authenticated user can view tickets owned by other users by guessing or enumerating ticket IDs. Semgrep did not flag this — it was identified via manual code review of authorization logic.

SOURCE FILE INFORMATION

File: `app.py`
Function: `view_ticket(ticket_id)`
Vulnerable Line: 95-98

SCREENSHOT OF SAST TOOL OUTPUT



```
app.py
>>> python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY
(( Blocking ))
Hardcoded variable `SECRET_KEY` detected. Use environment variables or config files instead
Details: https://sg.run/Ekde

20| app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"

>>> python.django.security.injection.code.user-eval.user-eval
(( Blocking ))
Found user data in a call to 'eval'. This is extremely dangerous because it can enable an attacker
to execute arbitrary remote code on the system. Instead, refactor your code to not use 'eval' and
instead use a safe library for the specific functionality you need.
Details: https://sg.run/PJDW

162| formula = request.args.get("formula", "1+1")
163| try:
164|     score = eval(formula)
165| except Exception as exc:
166|     return jsonify({"error": str(exc)}), 400

>>> python.flask.security.injection.user-eval.eval-injection
```

"Not detected by Semgrep — identified via manual code review."

SCREENSHOT OF VULNERABLE CODE

```

@app.route("/tickets/<int:ticket_id>")
def view_ticket(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    ticket = models.get_ticket(ticket_id)
    if not ticket:
        return "Ticket not found", 404
    comments = models.get_comments(ticket_id)
    return render_template("ticket.html", user=user, ticket=ticket, comments=comments)

```

WHY IS IT VULNERABLE?

The code checks authentication but not authorization:

```

@app.route("/tickets/<int:ticket_id>")
def view_ticket(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    ticket = models.get_ticket(ticket_id)
    if not ticket:
        return "Ticket not found", 404
    comments = models.get_comments(ticket_id)
    return render_template("ticket.html", user=user, ticket=ticket, comments=comments)

```

There is no check like:

```

if ticket["owner_id"] != user["id"]:
    return "Forbidden", 403

```

Ticket IDs are sequential integers (1, 2, 3...). An attacker can iterate through IDs to view tickets belonging to other users. Similarly, the `by_status()` and `search_tickets()` endpoints also return all tickets without filtering by owner.

This is a True Positive because the application has user ownership data available (`ticket.owner_id`, `user.id`) but does not enforce the check.

SECURITY IMPACT

Information Disclosure / Privacy Violation. An authenticated attacker can:

- View support tickets belonging to other users
- Read sensitive information disclosed in ticket descriptions
- Read internal comments on tickets
- Enumerate all tickets in the system

RECOMMENDED REMEDIATION

Add an ownership check before returning ticket data:

```
@app.route("/tickets/<int:ticket_id>")
def view_ticket(ticket_id):
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    ticket = models.get_ticket(ticket_id)
    if not ticket:
        return "Ticket not found", 404
    # Authorization check
    if ticket["owner_id"] != user["id"] and user["role"] != "admin":
        return "Forbidden", 403
    comments = models.get_comments(ticket_id)
    return render_template("ticket.html", user=user, ticket=ticket, comments=comments)
```

Apply the same ownership filter to search_tickets and by_status endpoints.

REFERENCES

- OWASP: Access Control Cheat Sheet
- CWE-639: Authorization Bypass Through User-Controlled Key
- CWE-862: Missing Authorization

FINDING 12

VULNERABILITY TITLE

Missing Admin Role Check on Admin Reports Page

CLASSIFICATION

True Positive

JUSTIFICATION

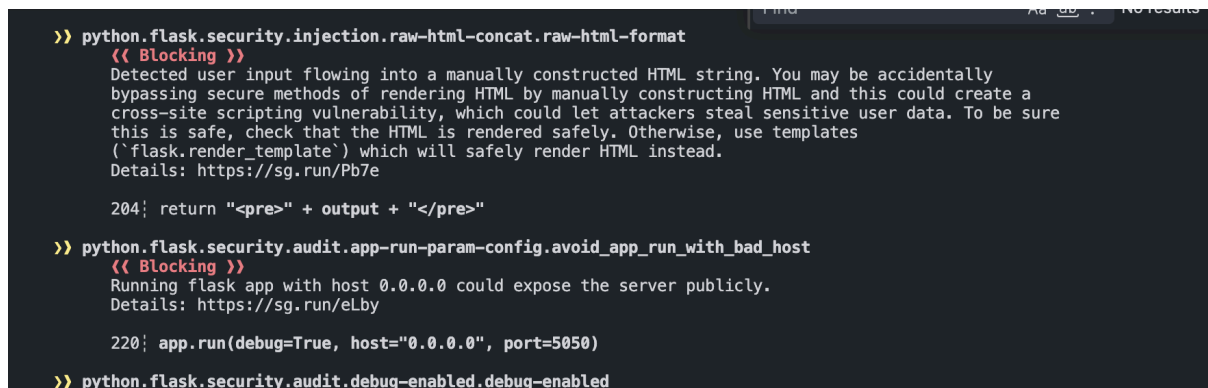
The /admin/reports endpoint at app.py:209-212 checks that the user is authenticated but does NOT verify that the user has the "admin" role.

Although the frontend template (base.html:17) only shows the admin link to admin users, the backend route itself has no authorization enforcement. Semgrep did not flag this — it was identified via manual code review.

SOURCE FILE INFORMATION

File: app.py
Function: admin_reports()
Vulnerable Line: 209-212

SCREENSHOT OF SAST TOOL OUTPUT



```
>> python.flask.security.injection.raw-html-concat.raw-html-format
(( Blocking ))
Detected user input flowing into a manually constructed HTML string. You may be accidentally
bypassing secure methods of rendering HTML by manually constructing HTML and this could create a
cross-site scripting vulnerability, which could let attackers steal sensitive user data. To be sure
this is safe, check that the HTML is rendered safely. Otherwise, use templates
('flask.render_template') which will safely render HTML instead.
Details: https://sg.run/Pb7e

204: return "<pre>" + output + "</pre>"

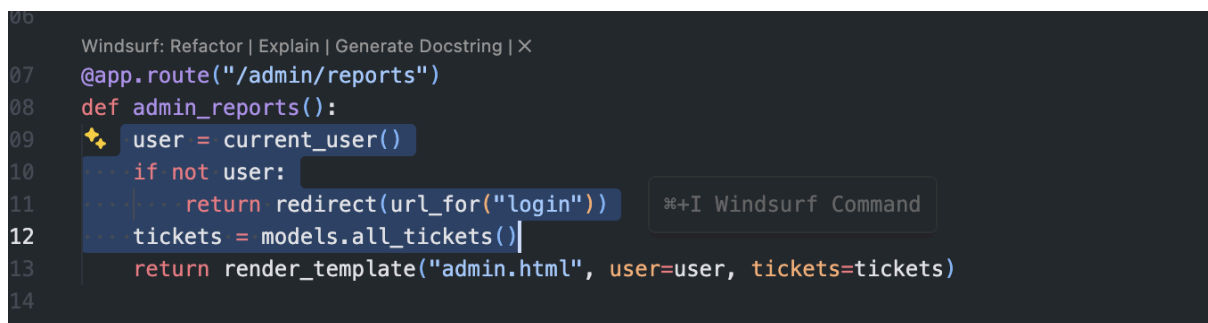
>> python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
(( Blocking ))
Running flask app with host 0.0.0.0 could expose the server publicly.
Details: https://sg.run/eLby

220: app.run(debug=True, host="0.0.0.0", port=5050)

>> python.flask.security.audit.debug-enabled.debug-enabled
```

"Not detected by Semgrep — identified via manual code review."

SCREENSHOT OF VULNERABLE CODE



```
06
Windsurf: Refactor | Explain | Generate Docstring | X
07 @app.route("/admin/reports")
08 def admin_reports():
09     user = current_user()
10     if not user:
11         return redirect(url_for("login"))
12     tickets = models.all_tickets()
13     return render_template("admin.html", user=user, tickets=tickets)
14
```

WHY IS IT VULNERABLE?

The admin_reports() function only checks authentication:

```
@app.route("/admin/reports")
def admin_reports():
    user = current_user()
    if not user:
        return redirect(url_for("login"))
    tickets = models.all_tickets()
    return render_template("admin.html", user=user, tickets=tickets)
```

There is no role check like:

```
if user["role"] != "admin":  
    return "Forbidden", 403
```

The frontend hides the admin link from non-admin users (base.html:17):

```
{% if user.role == 'admin' %}<a href="/admin/reports">Admin Reports</a>{% endif %}
```

However, this is only client-side hiding. Any authenticated user can navigate directly to /admin/reports in their browser and access the page. The admin page shows all tickets with owner usernames, which regular users should not see.

This is a True Positive because the security control relies entirely on client-side UI hiding with no server-side enforcement.

SECURITY IMPACT

Privilege Escalation / Information Disclosure. Any authenticated user can:

- View all tickets in the system (not just their own)
- See ticket ownership and usernames of all users
- Access sensitive data from other users' tickets

RECOMMENDED REMEDIATION

Add a role check at the beginning of the function:

```
@app.route("/admin/reports")  
def admin_reports():  
    user = current_user()  
    if not user:  
        return redirect(url_for("login"))  
    if user["role"] != "admin":  
        abort(403)  
    tickets = models.all_tickets()  
    return render_template("admin.html", user=user, tickets=tickets)
```

REFERENCES

- OWASP: Access Control Cheat Sheet
 - CWE-862: Missing Authorization
 - Principle of Least Privilege
-
-

FINDING 13

VULNERABILITY TITLE

SQL Injection in Tickets By Status Endpoint

CLASSIFICATION

False Positive

JUSTIFICATION

Semgrep flagged models.py:136 with formatted-sql-query and sqlalchemy-execute-raw-query because the SQL query uses .format() string interpolation. However, the status parameter is validated against a hardcoded whitelist (STATUS_COLUMNS dict at line 123-127) before it reaches the query. Only values "open", "closed", or "pending" are accepted — none of which can contain SQL injection characters. This is a False Positive.

SOURCE FILE INFORMATION

File: models.py
Function: tickets_by_status(status)
Vulnerable Line: 135-136 (flagged)
Mitigating Line: 131-133 (whitelist check)

SCREENSHOT OF SAST TOOL OUTPUT

```

models.py
>>> python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
(( Blocking ))
Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
ORM will be a better option.
Details: https://sg.run/2b1L

118| rows = conn.execute(sql).fetchall()

>>> python.lang.security.audit.formatted-sql-query.formatted-sql-query
(( Blocking ))
Detected possible formatted SQL query. Use parameterized queries instead.
Details: https://sg.run/EkWw

136| rows = conn.execute(sql).fetchall()

>>> python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
(( Blocking ))
Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
ORM will be a better option.
Details: https://sg.run/2b1L

136| rows = conn.execute(sql).fetchall()

```

SCREENSHOT OF VULNERABLE CODE

```

123 STATUS_COLUMNS = {
124     "open": "open",
125     "closed": "closed",
126     "pending": "pending",
127 }
128
129
Windsurf: Refactor | Explain | Generate Docstring | X
130 def tickets_by_status(status):
131     column = STATUS_COLUMNS.get(status)
132     if column is None:
133         return None
134     conn = get_connection()
135     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)"
136     rows = conn.execute(sql).fetchall()
137     conn.close()
138     return rows
139

```

WHY IS IT VULNERABLE?

The code builds a SQL query using `.format()`:

```

STATUS_COLUMNS = {
    "open": "open",
    "closed": "closed",
    "pending": "pending",
}

```

```

def tickets_by_status(status):
    column = STATUS_COLUMNS.get(status)

```



```
if column is None:
    return None
sql = "SELECT * FROM tickets WHERE status = '{}".format(column)
```

Semgrep sees the `.format()` on a SQL string and flags it as SQL injection. However, the whitelist at line 131 ensures column can ONLY be one of: "open", "closed", or "pending". If any other value is passed, the function returns None. These three values are hardcoded strings with no user input component.

Why is this a False Positive?

- The status parameter is validated against an allowlist before use
- Only three fixed, safe values can reach the SQL query
- Even if an attacker sends `status=anything_else`, the function returns None and the SQL query is never executed
- Semgrep performs static analysis and cannot evaluate that the whitelist is a complete and effective control

Note: While this pattern is safe, it is still better practice to use parameterized queries for defense-in-depth. The code is not wrong, just defensive coding could be improved.

SECURITY IMPACT

None (False Positive). The whitelist prevents any untrusted input from reaching the SQL query.

RECOMMENDED REMEDIATION

Since this is a False Positive, no remediation is strictly required. However, for consistency with other query patterns, consider using a parameterized query:

```
sql = "SELECT * FROM tickets WHERE status = ?"
rows = conn.execute(sql, (column,)).fetchall()
```

REFERENCES

- OWASP: Input Validation Cheat Sheet (Allowlisting)
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command

FINDING 14

VULNERABILITY TITLE

Missing CSRF Tokens in Forms

CLASSIFICATION

False Positive

JUSTIFICATION

Semgrep flagged templates/login.html:8 and templates/ticket.html:14,25,39 with django-no-csrf-token rule because the HTML forms do not include a CSRF token. However, this Django-specific rule is incorrectly applied to a Flask application. Flask does not have built-in CSRF protection like Django; CSRF protection in Flask requires a separate extension (Flask-WTF). The Semgrep rule is misconfigured — it targets Django templates but is scanning a Flask project. This is a False Positive due to incorrect rule context.

SOURCE FILE INFORMATION

File: templates/login.html, templates/ticket.html
Function: N/A (HTML templates)
Vulnerable Line: login.html:8, ticket.html:14, 25, 39

SCREENSHOT OF SAST TOOL OUTPUT

4 Code Findings

templates/login.html

>> python.django.security.django-no-csrf-token.django-no-csrf-token

((Blocking))

Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: <https://sg.run/N0Bp>

```
8: <form method="post">
9:   <div class="field">
10:     <label for="username">Username</label>
11:     <input type="text" id="username" name="username">
12:   </div>
13:   <div class="field">
14:     <label for="password">Password</label>
15:     <input type="password" id="password" name="password">
16:   </div>
17:   <button type="submit">Login</button>
[hid 1 additional lines, adjust with --max-lines-per-finding]
```

templates/ticket.html

>> python.django.security.django-no-csrf-token.django-no-csrf-token

((Blocking))

Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: <https://sg.run/N0Bp>

```
14: <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15:   <input type="hidden" name="cost" value="1">
16:   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17: </form>
-----
25: <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-
data">
26:   <input type="file" name="file">
27:   <button type="submit" class="btn-secondary">Upload attachment</button>
28: </form>
-----
39: <form method="post" action="/tickets/{{ ticket.id }}/comment">
40:   <div class="field">
41:     <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42:   </div>
43:   <button type="submit">Add comment</button>
44: </form>
```

SCREENSHOT OF VULNERABLE CODE

```

4   <div class="login-wrap">
5     <div class="card">
6       <h1>Sign in</h1>
7       {% if error %}<div class="error-banner">{{ error }}</div>{% endif %}
8     <form method="post">
9       <div class="field">
10        <label for="username">Username</label>
11        <input type="text" id="username" name="username">
12      </div>
13      <div class="field">
14        <label for="password">Password</label>
15        <input type="password" id="password" name="password">
16      </div>
17      <button type="submit">Login</button>
18    </form>
19  </div>

```

```

14  <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15    <input type="hidden" name="cost" value="1">
16    <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17  </form>
18 </div>
19
20 <div class="card">
21   <h2>Attachments</h2>
22   <ul class="attachment-list">
23     <li><a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt">boot_log.txt</a> &middot; <a href="/tickets/{{ ticket.id }}/
24   </ul>
25   <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-data">
26     <input type="file" name="file">
27     <button type="submit" class="btn-secondary">Upload attachment</button>
28   </form>
29 </div>
30
31 <div class="card">
32   <h2>Comments</h2>
33   {% for c in comments %}
34     <div class="comment">
35       <span class="comment-author">{{ c.author }}</span>{{ c.body|safe }}
36     </div>
37   {% endfor %}
38
39   <form method="post" action="/tickets/{{ ticket.id }}/comment">
40     <div class="field">

```

WHY IS IT VULNERABLE?

The forms in login.html and ticket.html use method="post" without including a CSRF token. The Semgrep django-no-csrf-token rule flags this as a CSRF vulnerability.

Why is this a False Positive?

- The rule is specific to Django's template system, which has built-in CSRF protection via {% csrf_token %}. Flask does not have the same mechanism.
- This is a Flask application (uses Flask, Jinja2 — not Django).
- While CSRF protection is still a valid concern for any web application, the Semgrep rule is marking Django-specific violations in a non-Django project. The rule has been applied outside its intended context.

Note: While this specific Semgrep finding is a False Positive, CSRF protection should still be added to this application as a separate security concern (see recommended remediation below). The classification

of "False Positive" applies to the Semgrep rule output specifically, not to the general security recommendation.

SECURITY IMPACT

None from this Semgrep finding alone (False Positive). However, the absence of CSRF protection in general means an attacker could potentially trick a logged-in user into submitting forms unknowingly (e.g., escalating a ticket, posting comments, uploading files). This is a separate design concern from the Semgrep rule misapplication.

RECOMMENDED REMEDIATION

Since the Semgrep finding is a False Positive (wrong framework), no remediation is required for the finding itself. However, for proper CSRF protection, install and configure Flask-WTF:

```
pip install Flask-WTF
```

Then in the application:

```
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)
```

And in templates:

```
<form method="post">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}" />
  ...
</form>
```

REFERENCES

- OWASP: Cross-Site Request Forgery Prevention Cheat Sheet
- CWE-352: Cross-Site Request Forgery (CSRF)
- Flask-WTF Documentation: CSRF Protection